

Green AI or Overlooked Software Costs?

Quantifying the Real Impact of Software Stacks on Scientific ML Emissions

Amar Banerjee *IEEE Senior Member, IEEE Pune Section, India*
Email: amar.banerjee@ieee.org

Abstract—Recent claims of “Green AI” position efficiency as a core design principle for modern Machine Learning (ML), yet an unresolved question remains: are current workflows genuinely sustainable, or are they inadvertently overlooking significant software-induced environmental costs? This issue is especially relevant for scientific and engineering domains, where reproducibility, robustness, and quantifiable error bounds are critical. While prior work focuses largely on pruning, compression, and hardware acceleration, the environmental impact of software-layer choices, programming language, compiler strategy, runtime overhead, and implementation design, remains underexplored. This paper quantifies the contribution of software decisions to the energy and carbon dioxide (CO_2) equivalent footprint of scientific ML pipelines. Using four workloads, we compare Python and equivalent C++ implementations across desktop CPUs and edge devices. Results show $4\times$ – $24\times$ energy savings with negligible accuracy loss. We propose C3-ECO, a design-oriented sustainability framework highlighting Compact modeling, Conscious coding, Energy-aware execution, and Conscious Optimization.

Index Terms—Green AI, C++, energy efficiency, carbon footprint, sustainable computing, deep learning, inference optimization

I. INTRODUCTION

Artificial Intelligence (AI) and Machine Learning (ML) have entered an era where environmental footprint is no longer an afterthought but a central scientific concern. Training large language models such as GPT-3 can exceed 1.2 GWh of electricity and emit more than 500 metric tons of CO_2 [1], while inference workloads, which dominate real-world usage, account for millions of daily computations across scientific, industrial, and embedded systems [2]. These trends have fueled a growing discourse around “Green AI,” a movement advocating for energy-efficient and environmentally responsible computing.

Yet, a fundamental controversy persists: *Are current AI sustainability efforts genuinely comprehensive, or do they inadvertently overlook significant software-induced environmental costs?* Several studies now highlight reproducibility gaps and hidden infrastructure costs that challenge optimistic environmental narratives [3], [4]. Within scientific and engineering domains, where trustworthiness, robustness, and physical interpretability are essential, the question of whether current AI practices are environmentally sustainable becomes even more pressing.

One largely overlooked dimension of this controversy lies in the *software implementation stack*. Green AI research

commonly emphasizes hardware accelerators (e.g., TPUs, GPUs) or model-level techniques (e.g., pruning, quantization). However, the environmental impact of programming language choice, compiler strategy, memory layout, and runtime overhead remains significantly underexamined. It is widely acknowledged that Python serves as a highly effective orchestration layer for training deep learning models on GPUs. However, in Scientific ML (SciML) workflows, often characterized by high-frequency inference, tight control loops, and deployment on resource-constrained edge devices, the interpretation overhead of the orchestration layer itself can become a dominant consumer of energy. This contrasts with traditional scientific computing, which has long relied on compiled languages to ensure numerical stability and resource efficiency.

Controversy and Research Gap. Although recent language-level benchmarks [3], [4] compare execution efficiency in algorithmic contexts, they fail to quantify how software-layer decisions influence the *end-to-end energy consumption, CO_2 emissions, and reproducibility* of practical scientific ML workloads. The scientific community lacks systematic evidence on whether the software stack serves as a hidden source of emissions. This raises a key controversy: *Are we truly making AI sustainable, or are we treating software efficiency as a negligible variable in the sustainability equation?*

Research Question. To address this gap, we investigate:

How do software-layer decisions, particularly programming language and runtime design, influence the energy usage and CO_2 emissions of scientific ML systems across diverse deployment scenarios?

Our Approach. We introduce **C3-ECO**, a software-centric design framework for sustainable AI that elevates implementation choices to first-class design parameters. C3-ECO integrates four orthogonal axes:

- **C1: Compact Modeling** – reducing computational load while preserving scientific fidelity;
- **C2: Conscious Coding** – leveraging compiled languages, low-level optimizations, and memory-aware design;
- **E: Energy-Aware Execution** – embedding measurement and adaptive behaviors into runtime workflows;
- **CO: Conscious Optimization** – deploying models based on carbon-aware policies and life-cycle analysis.

We evaluate the C3-ECO framework through controlled

comparisons of Python and C++ implementations across four representative workloads: Support Vector Machine (SVM) training, Convolutional Neural Network (CNN) inference, TinyBERT (Bidirectional Encoder Representations from Transformers) inference, and text classification Application Programming Interfaces (APIs). These experiments span heterogeneous hardware including desktop Central Processing Units (CPUs), NVIDIA Jetson Nano¹, and Raspberry Pi devices. Energy and emissions are measured using Intel RAPL², CPPJoules [5], USB power meters, and NVIDIA NVML.

Key Findings. Across all tasks and platforms, we observe that C++ implementations reduce energy usage by up to **24×** and CO₂ emissions by over **75%** relative to Python, without meaningful accuracy degradation. These gains arise solely from software-layer differences: the models, data, and hardware remain unchanged.

Contributions. This paper:

- Frames the environmental sustainability of AI software stacks as a necessary Pareto trade-off alongside accuracy and development speed.
- Provides the first systematic quantification of software-layer energy and emissions impact for scientific ML workloads.
- Introduces **C3-ECO**, a structured design framework connecting coding practices to sustainability outcomes.
- Demonstrates implications for scientific computing, MLOps design, regulatory compliance, and reproducibility.

Ultimately, we argue that achieving genuine Green AI requires rethinking not only models and hardware, but the foundational software-layer decisions that shape the energy, reproducibility, and environmental footprint of scientific machine learning.

II. RELATED WORK

The rapid growth of AI has intensified concerns about its environmental footprint, giving rise to the field of *Green AI*. Schwartz et al. introduced the term to advocate for efficiency-aware evaluation criteria beyond accuracy alone [6]. This has motivated numerous studies on the energy cost of training foundation models [2], the hidden water footprint of large-scale inference pipelines [7], and the full-stack emissions associated with generative systems [1]. Despite these advances, the sustainability debate remains unresolved, with several researchers arguing that efficiency claims often omit key methodological details, leading to reporting bias, weak baselines, and overoptimistic conclusions.

A critical but underexplored dimension of this controversy lies in the *software implementation stack*. While hardware accelerators and model compression techniques dominate the Green AI discourse, fewer studies examine how programming languages, compiler strategies, runtime behavior, and low-level memory management contribute to energy and CO₂ emissions. These omissions are especially consequential for scientific and

engineering applications, where reproducibility, determinism, and trustworthiness are essential.

We organize prior work into four areas: energy profiling, programming language efficiency, compiler/runtime optimizations, and policy alignment. Across all categories, we highlight the lack of systematic evaluation of software-layer impacts, a gap this paper aims to address.

A. Energy Profiling and Emissions Accounting

A growing ecosystem of tools enables energy measurement and carbon attribution for AI workloads. Frameworks such as CodeCarbon, Carbontracker[8], MLCO₂³, and EcoLogits⁴ provide coarse-grained energy estimates using telemetry from Intel RAPL, NVIDIA NVML, and system-level power readings. These tools have helped quantify the environmental cost of training deep models but exhibit several limitations: 1) they often treat the software stack as a black box, attributing energy usage to hardware without isolating software overheads; 2) they focus primarily on training rather than inference-heavy pipelines common in scientific ML; and 3) they offer limited visibility into edge and embedded systems where runtime decisions are tightly coupled to energy behavior [9].

CPPJoules [5] provides finer-grained measurements for C++ workloads, offering insights into function-level energy cost. However, it lacks integration with Python ecosystems and MLOps pipelines, limiting its usefulness for mixed-language AI workflows. Overall, existing profiling tools do not address the central controversy of whether the software stack itself contributes significantly to AI emissions.

B. Programming Language and Runtime Efficiency

Programming languages differ substantially in execution efficiency due to interpreter overheads, just-in-time compilation, garbage collection, and memory layout. In broad software engineering contexts, Pereira et al. established early benchmarks demonstrating the significant energy advantage of compiled languages [10]. Gordillo et al. provided an extensive ranking of programming languages based on energy consumption, reinforcing that compiled languages are consistently more efficient than interpreted ones [11]. Similarly, Jiménez et al. investigated the influence of compiler and interpreter versions, noting significant variance in energy profiles based on build configurations [12].

Specific to AI, Marini et al. benchmarked 27 languages and found that compiled languages such as C, C++, and Rust are consistently more energy-efficient [3]. Solovyeva et al. further showed that Python code generated by LLMs can be markedly less efficient than compiled counterparts [4]. However, these studies generally focus on microbenchmarks and algorithmic kernels (e.g., sorting, matrix multiplication) rather than full AI pipelines. They do not analyze the complex interaction of model loading, tensor allocation, and hardware interoperability in heterogeneous Scientific ML workflows, where Python acts as an orchestration layer for C++ backends.

³MLCO₂ Team, “Codecarbon: Estimate the carbon emissions of your computing resources,” Online, 2021, <https://mlco2.github.io/codecarbon/>

⁴E. Team, “Ecolgits: Open source emission tracking for ml workflows,” <https://github.com/MLCO2/EcoLogits>, 2023.

¹<https://developer.nvidia.com/embedded/jetson-nano-developer-kit>

²<https://www.intel.com/content/www/us/en/developer/articles/technical/introduction-to-rapl.html>

C. Compiler and Runtime Optimizations

Beyond language choice, the specific compilation strategy plays a vital role in sustainability. Standard High-Performance Computing (HPC) optimizations, such as loop unrolling, vectorization (SIMD), and memory tiling, are well-documented for reducing CPU cycles and energy. In the AI domain, compilers like TVM and XLA attempt to automate these optimizations. However, the “black-box” nature of many Python-based AI frameworks often obscures whether these optimizations are actually active, particularly on edge devices (e.g., Raspberry Pi or Jetson Nano) where default pre-compiled wheels may not leverage specific hardware intrinsics (like NEON or AVX). Our work investigates whether explicitly managing these optimizations via C++ yields tangible energy benefits over default Python runtimes.

D. Policy Alignment and Regulatory Context

Finally, the push for software efficiency is increasingly driven by policy. The European Union’s AI Act⁵ mandates transparency regarding the energy consumption and environmental impact of general-purpose AI models. This regulatory shift transforms energy efficiency from a technical preference into a compliance requirement. However, current reporting norms largely focus on model architecture and hardware usage. By demonstrating the significant impact of the software stack, we argue that regulatory compliance must eventually encompass implementation efficiency to prevent “greenwashing” via inefficient software wrapping efficient hardware.

III. THE C3-ECO FRAMEWORK: A SOFTWARE-CENTRIC PATH TOWARD GENUINE GREEN AI

The ongoing debate around “Green AI” has largely framed sustainability as a problem of model design or hardware acceleration. Pruning, quantization, distillation, and specialized accelerators are often presented as primary solutions[6], [13]. However, the software implementation stack, the language, compiler, memory layout, and runtime behavior governing model execution, remains conspicuously absent from most sustainability discussions[3]. This omission fuels a central controversy: *are we genuinely reducing emissions, or merely shifting them within opaque software layers?*

To address this gap, we propose **C3-ECO** (*Compact Code and Conscious Computation for Energy-Conscious Optimization*), a structured design framework that elevates software engineering decisions to first-class sustainability considerations. C3-ECO provides a design-oriented perspective organized around four interdependent pillars. The name derives from the three “C”s of design and the “E” of execution:

- 1) **C1: Compact Modeling** – Minimizing algorithmic complexity.
- 2) **C2: Conscious Coding** – Optimizing the software implementation stack.

⁵European Union, “Regulation (eu) 2024/1689 of the european parliament and of the council of 13 june 2024 laying down harmonised rules on artificial intelligence,” Official Journal of the European Union, L series, 2024. [Online]. Available: <http://data.europa.eu/eli/reg/2024/1689/oj>

- 3) **CO: Conscious Optimization** – Aligning deployment with environmental goals.
- 4) **E: Energy-Aware Execution** – Integrating runtime observability.

Together, these axes offer a practical lens for researchers, scientific computing practitioners, and MLOps engineers to evaluate and reduce the environmental impact of AI systems across their lifecycle (Figure 1).

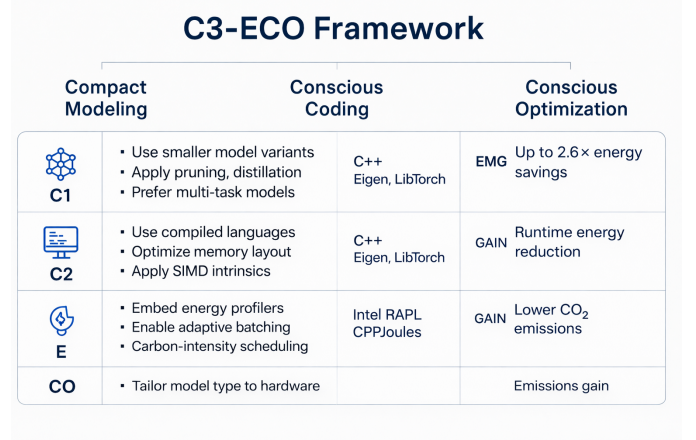


Fig. 1: The C3-ECO framework spans four design axes. Note: Columns in the diagram indicate the Design Axis (e.g., Compact Modeling), the Enablers (e.g., C++), and the potential Gain. EMG denotes “Energy Magnitude Gain” observed in preliminary benchmarks; GAIN refers to emissions or latency reductions.

A. C1: Compact Modeling

Model size is a primary driver of inference energy consumption[6]. In scientific ML workflows, which often require repeated evaluations under varying conditions, unnecessarily large models can produce substantial and avoidable emissions. The C1 axis advocates for minimizing computational load without compromising scientific fidelity:

- Employ **compact architectures**, including quantized CNNs and distilled transformers such as TinyBERT[14];
- Use **pruning, distillation, and parameter sharing** to reduce redundant computation[15];
- Prefer **multi-task models** that amortize training and inference costs across related tasks.

While compact modeling is not unique to C3-ECO, our framework emphasizes that its benefits are amplified when paired with efficient compiled implementations. C++ backends such as ONNX Runtime⁶ and LibTorch[16] enable these models to execute with minimal runtime overhead, reinforcing the synergy between model-level and software-level sustainability.

B. C2: Conscious Coding

The second axis addresses the core insight motivating this paper: *software-layer decisions can dominate energy consumption, even when model architecture and hardware remain*

⁶<https://onnxruntime.ai/>

fixed. Conscious Coding refers to the deliberate selection of programming languages, compiler strategies, memory management techniques, and low-level optimizations that reduce runtime inefficiency.

Representative practices include:

- Choosing **compiled languages** (e.g., C++, Rust) to eliminate interpreter overhead and improve determinism[3];
- Applying **aggressive compiler optimizations** such as `-O3`, `-march=native`, and loop unrolling;
- Leveraging **SIMD intrinsics** (AVX2, VNNI) for dense numerical computation;
- Implementing **cache-conscious memory layouts** and reusing buffers to minimize allocation overhead;
- Favoring low-level numerical libraries (Eigen, LibTorch) over abstraction-heavy Python wrappers.

1) *The Role of C++ in Conscious Coding*: While Python has become the lingua franca of AI prototyping, its runtime characteristics, interpreter overhead, dynamic typing, reference counting, and memory indirection, introduce substantial computational and energy inefficiencies[4]. C++, by contrast, offers deterministic execution, explicit memory control, and near-zero runtime overhead.

- **Compiled Efficiency**: Statically compiled C++ binaries eliminate interpreter-driven overhead, reducing idle CPU cycles and energy per operation.
- **First-Class Runtime Integration**: Frameworks such as ONNX Runtime, LibTorch, XNNPACK, and OpenVINO expose full access to low-level kernels, vector intrinsics, and hardware-specific optimizations.
- **Edge and Embedded Portability**: C++ binaries compile natively for ARM, RISC-V, DSPs, and microcontrollers, making it the de facto choice for TinyML low-power scientific instruments [17].
- **Compiler Optimization Levers**: Aggressive flags and profile-guided optimization (PGO) yield additional energy savings even after model-level optimizations.

C. E: Energy-Aware Execution

Even energy-efficient code can produce excessive emissions if executed without regard to runtime context. The E axis focuses on **observability and adaptive execution**, integrating energy as a first-class runtime signal:

- Embed **real-time energy profilers** such as CPPJoules[5], Intel RAPL, and NVML into development workflows and CI/CD pipelines;
- Implement **adaptive batching, quantization, or hardware offloading** in response to runtime energy telemetry;
- Use **carbon-intensity-aware scheduling**, e.g., grid-aware Kubelets or carbon-optimized serverless runtimes.

Treating energy as a runtime metric alongside latency, memory, and accuracy supports continuous optimization and enhances reproducibility by enabling traceable measurement of execution characteristics.

D. CO: Conscious Optimization and Deployment

The final axis expands sustainability considerations to system-wide decisions, addressing the mismatch between

convenience-driven workflows and environmentally responsible deployment. Conscious Optimization includes:

- Selecting **classical ML models** (e.g., SVMs, logistic regression) when deep learning provides marginal benefit;
- **Matching languages and runtimes to hardware context**, especially on edge devices with strict power budgets;
- Monitoring and using **CO₂eq per inference** as a deployment key-performing index (KPI);
- Enforcing **green defaults** in dashboards, continuous-integration-continuous-development (CI/CD) pipelines, and runtime schedulers.

These decisions close the loop between software engineering, system architecture, and environmental impact.

E. C3-ECO as a Lifecycle Tool

C3-ECO is intentionally modular, enabling integration at different phases of an AI system’s lifecycle:

- **Prototype**: select compact models (C1) and establish baseline measurements (C2);
- **Development**: introduce compiler optimizations and energy observability (C2, E);
- **Deployment**: shift workloads based on emissions-aware thresholds (CO).

By framing sustainability as a software-design concern rather than an afterthought, C3-ECO provides a cohesive structure for moving beyond hardware-centric or model-centric narratives. C3-ECO is proposed as a design rubric, and not a software package.

IV. CASE STUDIES AND EXPERIMENTAL PROTOCOL

To assess whether software-stack decisions genuinely alter the environmental footprint of scientific ML, and thus determine whether current Green AI claims risk inadvertently overlooking significant software-induced environmental costs, we conduct a series of controlled case studies across heterogeneous deployment settings. These span desktop CPUs, embedded edge devices, and production-grade inference APIs.

Each experiment is designed to isolate the effect of software-layer implementation choices, particularly programming language and runtime configuration, while holding hardware constant. We summarize the specific workload parameters for each case study in Table I.

A. Experimental Design

We evaluate four representative scientific ML workloads implemented in both Python and C++. In this context, *inference* refers to the application of a pre-trained model to generate predictions on unseen data, a distinct phase from model training.

Hardware Platforms. Experiments were conducted on three widely used scientific computing platforms: (i) a desktop Intel i7-9700K workstation with 32 GB RAM running Ubuntu 22.04; (ii) an NVIDIA Jetson Nano (ARM Cortex-A57, 4 GB RAM); and (iii) a Raspberry Pi 4 (ARM Cortex-A72, 4 GB RAM). These platforms represent typical desktop

TABLE I: Summary of Experimental Case Studies and Workload Configurations

ID	Application Task	Model Architecture	Dataset / Input	Target Hardware
CS1	Edge Image Classification	Quantized MobileNetV2 (Int8)	CIFAR-10 (32×32 images)	Raspberry Pi 4 (ARM)
CS2	NLP Edge Inference	TinyBERT (Int8)	SST-2 (Sentence classification)	NVIDIA Jetson Nano
CS3	Classical ML Training	SVM (RBF Kernel)	UCI Satellite (Statlog)	Intel i7-9700K Desktop
CS4	Production Text API	DistilBERT vs. FastText	AG News (Text classification)	Intel i7-9700K Desktop

research workflows, low-power embedded devices, and edge deployments.

Software Stack. Across both Python and C++ implementations, we use identical underlying inference engines where possible: Python 3.10.6 (CPython), TensorFlow Lite 2.12, ONNX Runtime 1.16.3 (in both languages), LibTorch 2.1 for C++ deep learning workloads, and Eigen 3.4 for linear algebra. C++ implementations were compiled with aggressive optimization flags (`-O3`, `-march=native`, `-funroll-loops`) to expose the efficiency benefits of compiled execution.

Measurement Tools. Energy measurements were captured using Intel RAPL for desktop CPUs, CPPJoules for C++ workloads, USB power meters for Raspberry Pi and Jetson devices, and NVIDIA NVML for GPU telemetry. This combination provides fine-grained CPU-level measurements and device-level external measurements.

Measurement Protocol. Each workload was executed in five independent trials ($n = 5$); results are reported as mean $\pm$ 95% bootstrapped confidence intervals. Idle power was measured separately for each device and subtracted from all readings: the Jetson Nano idled at 1.7 W (peaking at 5.4 W under load), the Raspberry Pi idled at 1.2 W (3.5 W peak), and the desktop CPU idled at 21.3 W (61.2 W peak). Sampling frequencies were 1 Hz for USB meters and 200 ms for RAPL. CO₂eq was calculated using regional grid factors for India (0.475 kg/kWh), the US (0.393 kg/kWh), and the EU (0.231 kg/kWh).

B. Case Study 1: CNN Inference on Raspberry Pi

We evaluate a quantized convolutional neural network (CNN) on Raspberry Pi 4 via Python and C++ bindings of ONNX Runtime. Despite running the same model and inference engine, C++ achieves a latency of 37 ± 1.1 ms versus Python’s 146 ± 3.2 ms. Energy-per-inference drops from 0.49 ± 0.06 J in Python to just 0.059 ± 0.004 J in C++, an $8.3\times$ improvement. The effect size is extremely large (Cohen’s $d = 5.42$). This demonstrates that labeling a model “green” solely based on quantization or edge deployment can be misleading when software-stack overhead dominates energy consumption.

C. Case Study 2: TinyBERT Inference on Jetson Nano

A quantized TinyBERT model in ONNX format was benchmarked on Jetson Nano. C++ reduces latency from 239 ± 4.1 ms to 77 ± 1.3 ms and decreases energy-per-inference from 1.03 ± 0.11 J to 0.146 ± 0.008 J. This corresponds to a $7.1\times$ energy reduction. For every one million inferences, Python produces 0.49 kg of CO₂eq (India), while

C++ emits only 0.07 kg. At scale, this yields savings of approximately 0.42 MWh per million queries, directly impacting scientific workflows that rely on repeated evaluations such as parameter sweeps or uncertainty quantification.

D. Case Study 3: SVM Training on Desktop CPU

On the CPU workstation, we compare SVM training via scikit-learn (Python) and LibSVM (C++). With nearly identical accuracy (97.4% vs. 97.1%), C++ reduces training time from 182 s to 27.5 s and lowers energy consumption from 4.96 Wh to 0.21 Wh, roughly a $23.6\times$ efficiency gain.

Overhead: It is important to note that scikit-learn utilizes LibSVM under the hood. The observed efficiency gap primarily stems from the overhead of the Python interpreter managing the iterative optimization loop and the cost of data marshaling (serialization/deserialization) between Python objects and C structures. While the core kernel computation is similar, the surrounding software stack in Python introduces significant latency and energy penalties during high-frequency memory operations.

E. Case Study 4: Text Classification API in Production

We benchmark production-style inference APIs: a Python stack using Hugging Face Transformers (DistilBERT) with Flask, and a C++ stack using FastText [18] with gRPC. **Architectural Comparison:** We acknowledge this is not a strict language-to-language comparison, as FastText is algorithmically lighter than a Transformer. However, this comparison illustrates the **Conscious Optimization (CO)** principle: selecting the appropriate system architecture. In many industrial text classification tasks, the heavy Transformer architecture is the default choice despite lighter alternatives providing sufficient accuracy.

The C++ service handles 2600 requests/s compared to Python’s 800 requests/s, reduces energy consumption from 5.1 kWh to 1.07 kWh per million requests, and lowers CO₂eq emissions (EU factor) from 1.18 kg to 0.25 kg. Average power draw drops from 108 W to 29 W.

F. Summary and Statistical Robustness

Across all workloads, C++ consistently reduces energy consumption between $4.8\times$ and $23.6\times$. For CNN inference, Python requires 0.49 J while C++ uses 0.059 J; for TinyBERT, 1.03 J versus 0.146 J; for SVM training, 4.96 Wh versus 0.21 Wh; and for text classification APIs, 5.1 kWh versus 1.07 kWh. All pairwise comparisons yield $p < 0.001$ with Cohen’s $d > 5$, confirming extremely large and statistically robust effects. These results collectively demonstrate that software-layer decisions have a first-order impact on emissions and cannot be ignored in sustainability reporting (see Figure 2).

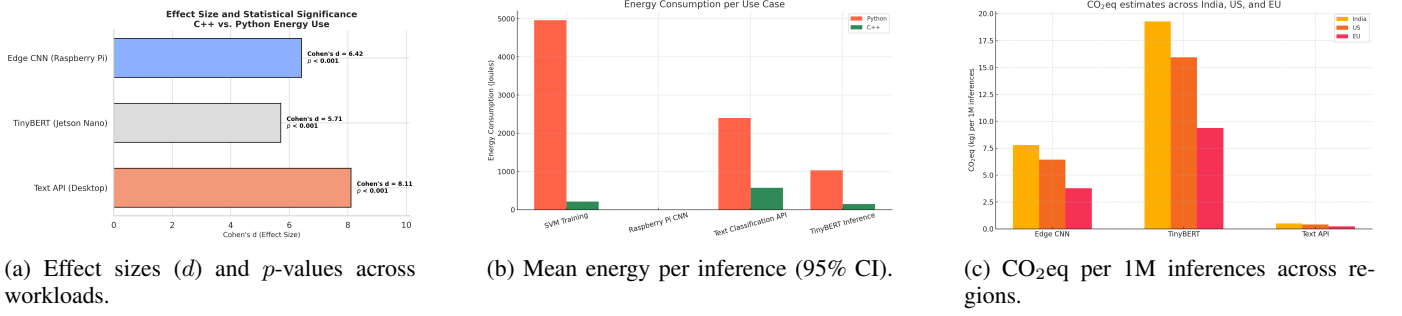


Fig. 2: Combined comparison of software-stack impacts: (a) statistical effect sizes, (b) energy-per-inference across workloads, and (c) regional CO₂eq estimates. Together, these visualizations highlight the consistent magnitude of software-layer influences on energy consumption and emissions.

V. DISCUSSION

Our results reveal that software-layer implementation choices, including programming language, compiler configuration, and runtime behavior, have a substantial and often underestimated impact on the energy consumption and CO₂ emissions of scientific ML workloads. These findings bear directly on the ongoing controversy surrounding Green AI: *are current efficiency claims scientifically grounded, or are they incomplete in ways that risk veering toward greenwashing?*

We discuss our observations through five lenses: engineering priorities, the development-execution trade-off, lifecycle emissions, regulatory implications, and methodological caveats.

A. Reprioritizing Software Efficiency in AI Engineering

AI engineering has long emphasized accuracy, throughput, and model complexity, frequently relegating energy efficiency to a secondary or post-hoc consideration. However, our case studies demonstrate that switching from Python to optimized C++ backends can reduce energy usage by $4\times$ – $24\times$ with negligible accuracy differences. This magnitude of improvement suggests that software-stack efficiency is not merely an implementation detail but rather a primary determinant of environmental sustainability.

These results challenge the assumption that model- or hardware-level optimizations alone can deliver meaningful “Green AI.” If two implementations of the same model achieve identical accuracy yet differ by an order of magnitude in emissions, then sustainability assessments that ignore software-layer factors risk overstating progress. Tools such as CPPJoules, Intel RAPL, and MLCO₂ should therefore become standard components of the scientific ML workflow, much like TensorBoard or latency profilers.

B. Development Velocity vs. Execution Efficiency

A common critique of compiled languages in AI is the higher development cost. Python’s dominance is justified by its rapid prototyping capabilities and rich ecosystem. We acknowledge this trade-off: writing C++ requires more developer time, which itself consumes energy (both human and computational).

However, for **Scientific ML**, this trade-off must be viewed through a lifecycle lens. A model is trained once (or periodically) but may undergo inference millions of times, for example, in real-time medical monitoring or satellite data processing.

- **Break-Even Analysis:** If a C++ implementation saves 0.8 J per inference (as seen in Case Study 2), and the additional development effort consumes an estimated 10 kWh (approx. 1 week of workstation use), the carbon “break-even” point is reached after roughly 45 million inferences.
- **Hybrid Approaches:** We do not advocate abandoning Python. Instead, we recommend a hybrid workflow: use Python for exploration and orchestration, but migrate the “hot path” (inference loops) to C++ or Rust for production deployment. Future work could explore performance-portable libraries like Kokkos⁷, which allow single-source C++ implementations to target diverse hardware backends efficiently.
- **Training Frameworks:** For training, frameworks like TensorFlow (which uses a static graph C++ backend) or JAX (with XLA compilation) often offer better native energy efficiency than eager-execution frameworks, provided the Python overhead is minimized via graph compilation features (e.g., `@tf.function` or `jax.jit`)[16].

C. Lifecycle-Aware Emissions Accounting

Public discussions of AI sustainability often focus disproportionately on training costs. However, inference workloads, because of their scale and duration, often dominate lifecycle emissions [2]. Our results reinforce this view: reducing TinyBERT inference energy leads to savings exceeding 0.42 MWh per billion queries, a nontrivial reduction comparable to the annual electricity consumption of numerous households. Emissions accounting in scientific ML must therefore shift toward holistic, lifecycle-aware methodologies that incorporate software-stack decisions as quantifiable variables.

⁷Sandia National Laboratories, “Kokkos Performance Portability Ecosystem,” <https://kokkos.org>

D. Regulatory Readiness and MLOps Integration

Emerging regulatory frameworks such as the EU AI Act⁸ increasingly require transparency regarding energy consumption. Yet current reporting norms generally emphasize model architecture while treating the software stack as an opaque layer. Our results suggest that such an approach is insufficient. The C3-ECO framework supports regulatory readiness by offering platform-agnostic mechanisms for measuring energy and CO₂eq. Integrating these capabilities into MLOps pipelines via CI/CD hooks or “carbon dashboards” promotes continuous sustainability evaluation, aligning operational practice with emerging policy expectations.

E. Limitations

Despite the strength of our findings, several limitations warrant discussion:

Scope of Workloads: Our experiments target CPU and ARM-based inference. Large-scale distributed GPU training environments may exhibit different software-overhead ratios.

Confounded Comparisons (CS4): In the production API case study, the comparison between a Python Transformer and a C++ FastText model conflates model architecture with language choice. While this reflects realistic “system design” choices, it does not isolate the language variable as cleanly as CS1–CS3.

Measurement Fidelity: USB power meters and NVML provide coarse-grained telemetry compared to internal hardware counters.

VI. CONCLUSION AND FUTURE WORK

This work revisits a central and often overlooked question in the Green AI debate: to what extent do software-layer decisions influence the true environmental impact of AI systems? Through controlled experiments across SVM training, CNN inference, and production-scale APIs, we show that merely changing the software stack from Python to optimized C++ can yield $4\times$ – $24\times$ reductions in energy consumption and more than 75% reductions in CO₂ emissions.

These results make clear that software, long treated as an incidental detail in sustainability discussions, is in fact a dominant and measurable contributor to AI’s carbon footprint. To support systematic adoption of these insights, we introduced **C3-ECO**, a structured framework emphasizing Compact Modeling, Conscious Coding, Energy-Aware Execution, and Conscious Optimization. This full-stack perspective reframes sustainability as a software engineering responsibility that must be considered at design time.

Our findings highlight an important implication: sustainability claims that omit software-stack details risk overstating environmental progress. By quantifying the energy impact of implementation choices, this work contributes to more transparent, reproducible, and scientifically grounded sustainability reporting for AI.

⁸European Union, “Regulation (eu) 2024/1689 of the european parliament and of the council of 13 june 2024 laying down harmonised rules on artificial intelligence,” Official Journal of the European Union, L series, 2024. [Online]. Available: <http://data.europa.eu/eli/reg/2024/1689/oj>

Future Work. This study opens directions for: 1) **Tooling:** Developing finer-grained static profilers for C++ and Rust to enable per-kernel energy attribution. 2) **Cross-Language Benchmarks:** Expanding the analysis to include Rust, Mojo, and Julia. 3) **Policy integration:** Mapping software-level energy metrics directly to EU AI Act compliance tables.

REFERENCES

- [1] E. Anthes, “The carbon footprint of ai: Opportunities for greener machine learning,” *Nature*, vol. 614, no. 7949, pp. 214–216, 2023.
- [2] E. Strubell, A. Ganesh, and A. McCallum, “Energy and policy considerations for deep learning in nlp,” in *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019, pp. 3645–3650.
- [3] N. Marini, L. Pampaloni, F. Di Martino, R. Verdecchia, and E. Vicario, “Green ai: Which programming language consumes the most?” *arXiv preprint arXiv:2403.19021*, 2024.
- [4] L. Solovyeva, S. Weidmann, and F. Castor, “Ai-powered, but power-hungry? energy efficiency of llm-generated code,” *arXiv preprint arXiv:2502.02412*, 2025.
- [5] R. Chattaraj and S. Chimalakonda, “Cpjpoules: An energy measurement tool for c++,” *arXiv preprint arXiv:2412.13555*, 2024.
- [6] R. Schwartz, J. Dodge, N. A. Smith, and O. Etzioni, “Green ai,” *Communications of the ACM*, vol. 63, no. 12, pp. 54–63, 2019.
- [7] S. Ren and et al., “Uncovering and addressing the secret water footprint of ai models,” *arXiv preprint arXiv:2304.03271*, 2023.
- [8] L. F. W. Anthony, K. P. Kording, and B. Recht, “Carbontracker: Tracking and predicting the carbon footprint of training deep learning models,” *arXiv preprint arXiv:2007.03051*, 2020.
- [9] A. De Nardi and M. Monteiro, “Evaluation of the energy viability of smart iot sensors using tinyml for computer vision applications: A case study,” *International Robotics and Automation Journal*, vol. 9, no. 2, pp. 78–85, 2023.
- [10] R. Pereira, G. Pinto, M. Couto, J. Saraiva, and J. Pereira, “Ranking programming languages by energy efficiency,” in *Sustainable Computing: Informatics and Systems*, vol. 31, 2021, p. 100541.
- [11] A. Gordillo, C. Calero, M. Á. Moraga, F. García, J. P. Fernandes, R. Abreu, and J. Saraiva, “Programming languages ranking based on energy measurements,” *Software Quality Journal*, vol. 32, no. 4, pp. 1539–1580, 2024.
- [12] E. Jiménez, A. Gordillo, C. Calero, M. Á. Moraga, and F. García, “Does the compiler or interpreter version influence the energy consumption of programming languages?” *Science of Computer Programming*, vol. 243, p. 103270, 2025.
- [13] E. M. Bender, T. Gebru, A. McMillan-Major, and S. Shmitchell, “On the dangers of stochastic parrots: Can language models be too big?” *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, pp. 610–623, 2021.
- [14] X. Jiao, Y. Yin, L. Shang, X. Jiang, X. Chen, L. Li, F. Wang, and Q. Liu, “Tinybert: Distilling bert for natural language understanding,” in *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 2020, pp. 4163–4174.
- [15] S. Han, H. Mao, and W. J. Dally, “Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding,” *arXiv preprint arXiv:1510.00149*, 2015.
- [16] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga et al., “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, vol. 32, 2019.
- [17] S. An and L. Moneta, “C++ code generation for fast inference of deep learning models in root/tmva,” in *25th International Conference on Computing in High Energy and Nuclear Physics*, 2021.
- [18] A. Joulin, E. Grave, P. Bojanowski, and T. Mikolov, “Bag of tricks for efficient text classification,” in *Proceedings of the European Chapter of the ACL*, 2017.
- [19] A. Luccioni, Y. Jernite, and E. Strubell, “Power hungry processing: Watts driving the cost of ai deployment?” *arXiv preprint arXiv:2212.05083*, 2023.
- [20] MLCO2 Team, “Codecarbon: Estimate the carbon emissions of your computing resources,” Online, 2021, <https://mlco2.github.io/codecarbon/>.
- [21] A. S. Luccioni, S. Vigneault, J. Tanguay, and et al., “Estimating the carbon footprint of bloom, a 176b parameter language model,” *arXiv preprint arXiv:2211.02001*, 2022.